

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, Decana de América)

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingeniería de Software



Curso: Base de Datos II

Sección: 02

Asunto: Informe técnico “Fundación Educa Perú”

Alumnos:

- Benites Meza, Marco Fabricio.
- Chávez Gave, José Luis.
- Dávalos Benito, Rodrigo.
- Lozano Paredes, Renzo Jeanpier Santos.
- Munayco Vivanco, Renzo Alexander.
- Torres Mariluz, Josué Armando.

Docente: Mg. Jorge Luis Chávez Soto.

Lima, 2025

1. Contexto y Problema

El "Programa de Becas Educativas" es una iniciativa social diseñada para mitigar las barreras económicas que enfrentan estudiantes en situación de vulnerabilidad en el Perú. Actualmente, muchos jóvenes con talento se ven forzados a abandonar sus estudios debido a limitaciones financieras, lo cual restringe tanto su desarrollo personal como el crecimiento profesional de sus comunidades. La Fundación Educa Perú busca conectar a estos estudiantes con donantes e instituciones educativas para brindar igualdad de oportunidades.

Alcance del Sistema de Base de Datos:

El sistema abarca la gestión integral del ciclo de vida de la beca, desde la postulación hasta el seguimiento académico y financiero.

- **Alcance:** Incluye los módulos de postulación y registro, evaluación de requisitos, asignación de becas basada en fondos disponibles, gestión de donantes y financiamiento, ejecución de pagos y auditoría del sistema.
- **Limitaciones:** El sistema se centra en la gestión de la información y reglas de negocio (validaciones, integridad referencial); no gestiona directamente las transacciones bancarias externas, aunque registra los desembolsos. La implementación física actual está optimizada para Oracle Database.

2. Objetivos Técnicos

Objetivo General:

Implementar una base de datos robusta y escalable para el Sistema "Fundación Educa Perú" que garantice la integridad, seguridad y trazabilidad de la información relacionada con la gestión de becas educativas.

Objetivos Específicos:

1. Automatizar el registro y evaluación: Gestionar eficientemente las postulaciones y la validación de requisitos socioeconómicos y académicos.
2. Controlar la asignación financiera: Asegurar que toda beca asignada cuente con respaldo financiero de un donante específico (integridad referencial entre financiamiento y beca).
3. Monitoreo Académico: Permitir el seguimiento del rendimiento de los becarios para aplicar reglas de renovación o suspensión de beneficios.

4. Trazabilidad y Auditoría: Registrar operaciones críticas y cambios de estado para garantizar la transparencia del proceso.

3. *Procesos de Negocio Principales*

a. Proceso de Postulación

- **Descripción:** El estudiante registra sus datos y sube evidencias. El sistema valida reglas de unicidad y completitud.
- **Actores:** Estudiante (EST), Sistema.
- **Entradas:** Datos personales, DNI, documentos socioeconómicos.
- **Salidas:** Registro de postulación con estado "Enviada" o "Observada".
- **Tablas Impactadas:** EDU_ESTUDIANTE, EDU_POSTULACION.

b. Proceso de Financiamiento

- **Descripción:** Registro de los aportes realizados por donantes para programas de becas específicos.
- **Actores:** Donante, Administrador.
- **Entradas:** Datos del donante, monto del compromiso, programa destino.
- **Salidas:** Saldo disponible para asignar becas.
- **Tablas Impactadas:** EDU_DONANTE, EDU_FINANCIAMIENTO, EDU_PROGRAMA_BECA.

c. Proceso de Asignación de Beca

- **Descripción:** Vinculación de un estudiante apto con un programa de beca que tenga fondos disponibles.
- **Actores:** Coordinador de Becas (CB), Sistema.
- **Entradas:** Postulación aprobada, ID de financiamiento disponible.
- **Resultados:** Beca activa y cronograma de pagos generado.
- **Tablas Impactadas:** EDU_BECA_ASIGNADA, EDU_FINANCIAMIENTO (actualización de saldo).

d. Proceso de Pago (Desembolso)

- **Descripción:** Ejecución de los beneficios económicos periódicos a los becarios activos.
- **Actores:** Tesorería, Sistema.
- **Entradas:** Beca activa, fecha programada.
- **Resultados:** Registro de pago ejecutado, comprobante.
- **Tablas Impactadas:** EDU_PAGO, EDU_BECA_ASIGNADA.

e. Proceso de Seguimiento Académico

- **Descripción:** Monitoreo periódico de notas y cumplimiento de requisitos para mantener la beca.

- **Actores:** Tutor, Sistema.
- **Entradas:** Promedios, constancias de matrícula.
- **Resultados:** Alertas de riesgo, renovación o suspensión de la beca.
- **Tablas Impactadas:** EDU_SEGUIMIENTO, EDU_ESTUDIANTE.

4. *Tabla de Reglas de Negocio (RN)*

Esta tabla consolida las reglas mencionadas en los entregables, vinculándolas con las tablas del modelo físico.

Tabla 01: Reglas de Negocio del sistema “Fundación Educa Perú”

ID Regla	Descripción de la Regla	Proceso Asociado	Implementación (Tablas/Columnas)
RN-23	Toda asignación de beca debe estar condicionada a la existencia de fondos disponibles.	Financiamiento / Asignación	EDU_FINANCIAMIENTO (Validación de saldo disponible).
RN-40 a RN-43	Reglas de renovación y suspensión basadas en promedios y créditos acumulados.	Seguimiento Académico	EDU_SEGUIMIENTO (Promedio, Estado Académico).
RN-61	No se puede registrar un pago sin una beca asignada existente; permite reportes de ejecución.	Pago	EDU_PAGO.ID_BECA (FK hacia Beca Asignada).
RN-Unicidad	Un estudiante no puede tener múltiples postulaciones activas al mismo programa.	Postulación	EDU_POSTULACION (PK Compuesta: ID_EST, ID_PROG).
RN-Trazabilidad	Las operaciones críticas deben	Auditoría	EDU_AUDITORIA (Trigger o

	registrar entidad afectada, usuario, fecha y valores antes/después.		Procedimiento).
--	---	--	-----------------

5. *Modelo de Datos Conceptual*

El modelo de datos conceptual del sistema “Fundación Educa Perú” representa, a nivel de negocio, las entidades clave y las relaciones que permiten gestionar el ciclo de vida de las becas: desde la postulación del estudiante, el financiamiento aportado por los donantes, hasta el seguimiento académico del becario y la trazabilidad de las operaciones.

Este modelo se ha construido a partir de los procesos definidos para la fundación y de las entidades ya identificadas en el modelo lógico (student, scholarship_program, call, application, donor, funding, scholarship_assigned, payment, academic_tracking, audit_log, etc.).

5.1. **Entidades del Modelo Conceptual**

En el modelo conceptual se identifican las siguientes entidades principales (se muestran con sus nombres en español para el informe, manteniendo la correspondencia con el modelo lógico y físico):

1. **Estudiante**

Representa al postulante o becario. Almacena sus datos personales, académicos y socioeconómicos, necesarios para evaluar su elegibilidad y realizar el seguimiento de su avance.

2. **Institución educativa**

Registra los colegios, institutos o universidades donde estudian los postulantes y becarios. Permite segmentar a los estudiantes por tipo de institución y localización.

3. **Tipo de beca**

Define la categoría de la beca (total, parcial, manutención, etc.), así como las reglas generales asociadas al tipo (por ejemplo, si cubre matrícula, pensión, alimentación, materiales, etc.).

4. Programa de beca

Representa los programas de becas específicos que ofrece la fundación, con sus requisitos de postulación, montos máximos financiables y alcance (nivel escolar, región, modalidad).

5. Convocatoria

Define las convocatorias que se abren para que los estudiantes puedan postular a un programa de beca, indicando fechas de inicio y cierre, programa asociado y condiciones particulares.

6. Postulación

Registra la solicitud de beca presentada por un estudiante en el marco de una convocatoria. Incluye datos como fecha de registro, estado (borrador, enviada, admitida, no admitida) y resultados de evaluación.

7. Beca asignada

Representa la beca finalmente otorgada a un estudiante, derivada de una postulación aceptada. Contiene monto mensual o total, duración, periodo de vigencia, condiciones y estado (activa, suspendida, finalizada).

8. Pago / desembolso

Registra los pagos programados y ejecutados al becario (por ejemplo, mensualidades de la beca). Incluye monto, periodo, fecha programada, fecha de ejecución y estado del pago.

9. Donante

Agrupar a las personas, empresas u organizaciones que financian los programas de becas. Contiene datos de identificación, contacto y el compromiso económico asumido.

10. Financiamiento

Registra los aportes económicos específicos que realiza cada donante hacia uno o varios programas de beca, definiendo el monto, fecha y saldo disponible. En el modelo físico esta entidad soporta la relación entre donante y programa de beca.

11. Seguimiento académico

Permite llevar el control periódico del rendimiento académico y/o socioemocional del becario, por periodo académico. Incluye promedio, créditos matriculados y aprobados, asistencias, alertas y planes de mejora.

12. Registro de auditoría

Registra operaciones críticas realizadas sobre las entidades del sistema (altas, modificaciones, bajas), para fines de trazabilidad y control de calidad. Almacena la entidad afectada, usuario, fecha y los valores antes/después del cambio.

5.2. Relaciones y cardinalidades del modelo conceptual

A nivel conceptual, las entidades anteriores se relacionan de la siguiente manera, reflejando las reglas de negocio del sistema “Fundación Educa Perú”:

1. Institución educativa – Estudiante

- **Cardinalidad:** Una institución educativa puede tener muchos estudiantes; cada estudiante pertenece a una sola institución (1:N).
- **Descripción:** Permite conocer en qué institución está matriculado el postulante o becario y segmentar la población beneficiaria.

2. Tipo de beca – Programa de beca

- **Cardinalidad:** Un tipo de beca puede estar asociado a varios programas de beca; cada programa está clasificado en un solo tipo (1:N).
- **Descripción:** Garantiza que los programas hereden las características generales del tipo de beca (total, parcial, manutención, etc.).

3. Programa de beca – Convocatoria

- **Cardinalidad:** Un programa de beca puede generar varias convocatorias en el tiempo; cada convocatoria pertenece a un solo programa (1:N).
- **Descripción:** Permite abrir sucesivas convocatorias para un mismo programa, sin duplicar su definición.

4. Estudiante – Postulación – Convocatoria

- **Cardinalidad conceptual:**

- Un estudiante puede registrar muchas postulaciones a diferentes convocatorias (1:N).
- Una convocatoria puede recibir muchas postulaciones de múltiples estudiantes (1:N).
- **Descripción:** La entidad **Postulación** actúa como entidad asociativa que vincula al estudiante con una convocatoria, añadiendo atributos propios como fecha, estado y resultado de evaluación.

5. Postulación – Beca asignada

- **Cardinalidad:** Cada postulación aceptada genera a lo sumo una beca asignada (0..1:1); cada beca asignada proviene de una única postulación.
- **Descripción:** Asegura la trazabilidad entre la solicitud original y la beca finalmente otorgada.

6. Donante – Financiamiento – Programa de beca

- **Cardinalidad conceptual:**
 - Un donante puede financiar varios programas de beca.
 - Un programa de beca puede recibir financiamiento de múltiples donantes.
- **Descripción:** La entidad **Financiamiento** modela esta relación N:M, registrando el detalle de cada aporte (monto, fecha, saldo), y garantizando que las asignaciones de becas estén respaldadas por fondos disponibles.

7. Beca asignada – Pago

- **Cardinalidad:** Una beca asignada se ejecuta a través de muchos pagos o desembolsos; cada pago corresponde a una sola beca asignada (1:N).
- **Descripción:** Permite controlar la ejecución financiera del programa por periodos (mensual, bimestral, etc.) y generar reportes de pagos realizados.

8. Estudiante / Beca asignada – Seguimiento académico

- **Cardinalidad:**
 - Un estudiante puede tener múltiples registros de seguimiento académico a lo largo de su trayectoria (1:N).
 - Opcionalmente, cada registro de seguimiento puede asociarse también a una beca asignada específica (por ejemplo, cuando el estudiante tiene más de una beca en momentos distintos).

- **Descripción:** Esta relación recoge el rendimiento y comportamiento del becario por periodo, y sirve como insumo para decisiones de renovación, suspensión o cierre de la beca.

9. Registro de auditoría – Entidades de negocio

- **Cardinalidad conceptual:** Una operación sobre cualquier entidad de negocio (estudiante, postulación, beca asignada, pago, financiamiento, etc.) puede generar uno o varios registros de auditoría; cada registro de auditoría describe una única operación.
- **Descripción:** A nivel conceptual, la entidad Registro de auditoría se relaciona con el resto a través de un atributo que identifica la entidad afectada y su identificador, proporcionando trazabilidad e historial de cambios.

6. *Modelo de Datos Lógico*

El modelo de datos lógico del sistema “Fundación Educa Perú” se diseñó siguiendo el paradigma relacional. En él se identifican las entidades principales (estudiantes, programas de beca, convocatorias, postulaciones, donantes, financiamiento, seguimiento, etc.) y sus relaciones, de acuerdo a los procesos de negocio definidos para la fundación.

Tabla 02: Modelo de Datos Lógico del Sistema “Fundación Educa Perú”.

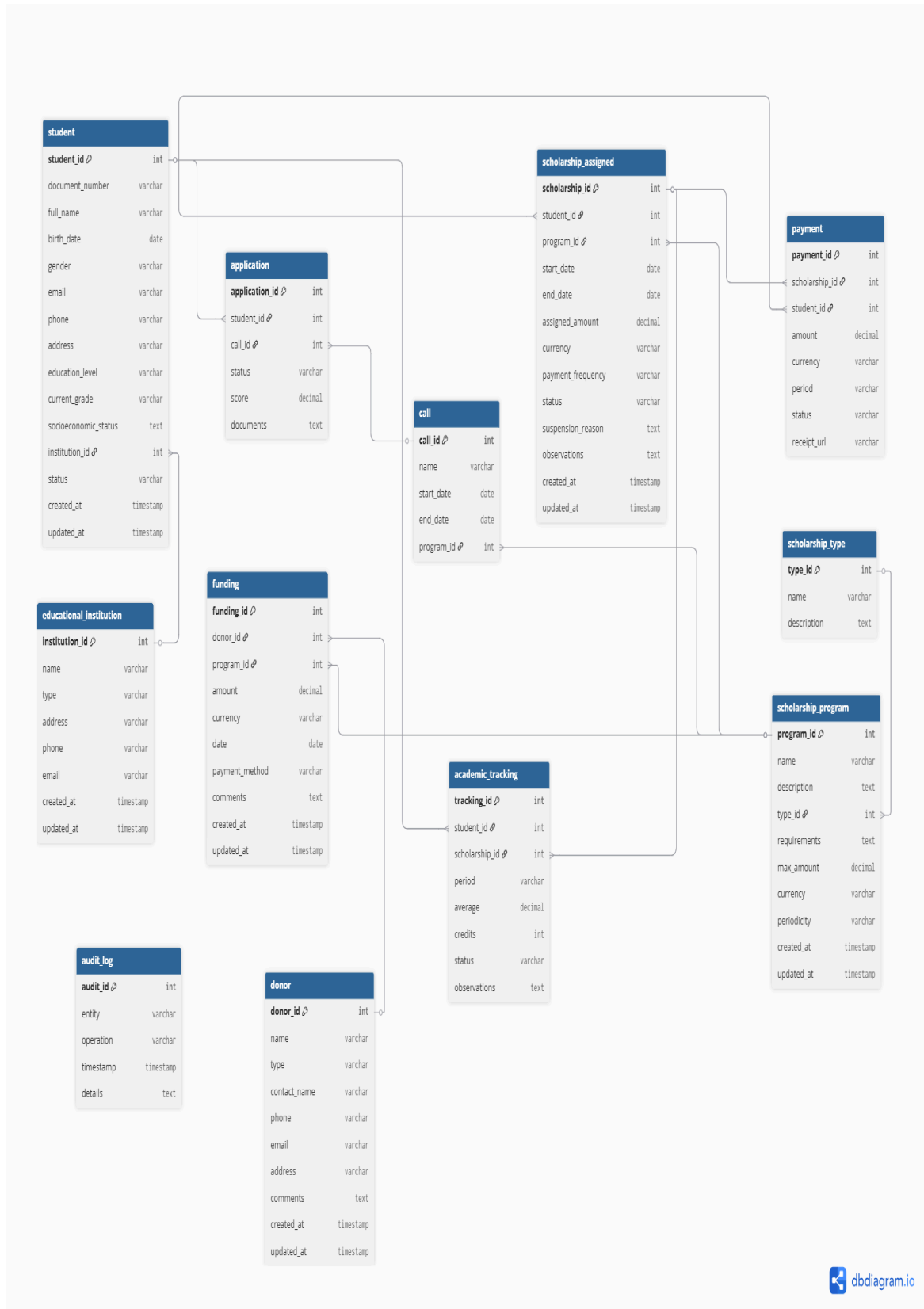
Nº	Entidad	Descripción Funcional
1	student	Registra los datos personales, académicos y socioeconómicos del postulante o becario.
2	educational_intitution	Almacena información de colegios, institutos o universidades donde estudian los becarios.
3	scholarship_type	Define los tipos de beca disponibles (total, parcial, manutención, etc.).

4	scholarship_program	Representa los programas de becas ofrecidos por la fundación, con requisitos y montos máximos.
5	call	Define las convocatorias activas para postulación, con fechas y programa asociado.
6	application	Registra las solicitudes de beca enviadas por los estudiantes en una convocatoria específica.
7	scholarship_assigned	Representa la beca otorgada a un estudiante, incluyendo monto, duración y estado.
8	payment	Controla los desembolsos realizados a los becarios, con fecha, monto y estado.
9	donor	Almacena los datos de personas, empresas u ONGs que financian becas.
10	funding	Registra los aportes económicos de los donantes hacia programas específicos.
11	academic tracking	Permite hacer seguimiento al rendimiento académico del becario por periodo.
12	audit_log	Registra operaciones críticas realizadas en el sistema para fines de trazabilidad y control.

Enlace del Diagrama del Modelo de Datos Lógico del Proyecto de BD2:

<https://dbdiagram.io/d/690924926735e111700dd76>

Diagrama del Modelo de Datos Lógico:



7. *Modelo de Datos Físico*

El Modelo de Datos Físico (MDF) presentado a continuación ha sido diseñado e implementado sobre Oracle Database. Este modelo traduce las entidades, relaciones y reglas de negocio del diseño lógico en una estructura optimizada y concreta, utilizando tipos de datos específicos, restricciones de integridad y mecanismos de trazabilidad, asegurando así la robustez y el rendimiento del Sistema de Gestión del Programa de Becas Educativas.

Tabla 03: Modelo de Datos Físico del “Sistema Fundación Educa Perú”.

Tabla	Rol en el Negocio	Implementación Física Destacada
EDU_ESTUDIANTE	Almacena los datos personales, académicos y socioeconómicos del beneficiario.	Contiene claves foráneas (FK_EST_EDI_INSTITUCION) para segmentar por institución.
EDU_POSTULACION	Registra el ciclo de vida de la solicitud (desde el estado 'borrador' hasta 'admisible' o 'no admisible').	Su clave primaria compuesta (PK_POST_EST_...) asegura la unicidad de las postulaciones por estudiante y programa.
EDU_SEGUIMIENTO	Monitorea el rendimiento académico y socioemocional del becario, activando alertas y planes de mejora.	Diseñada para el registro periódico (periodo) de promedios y créditos, crucial para las reglas de renovación y suspensión (RN-40 a

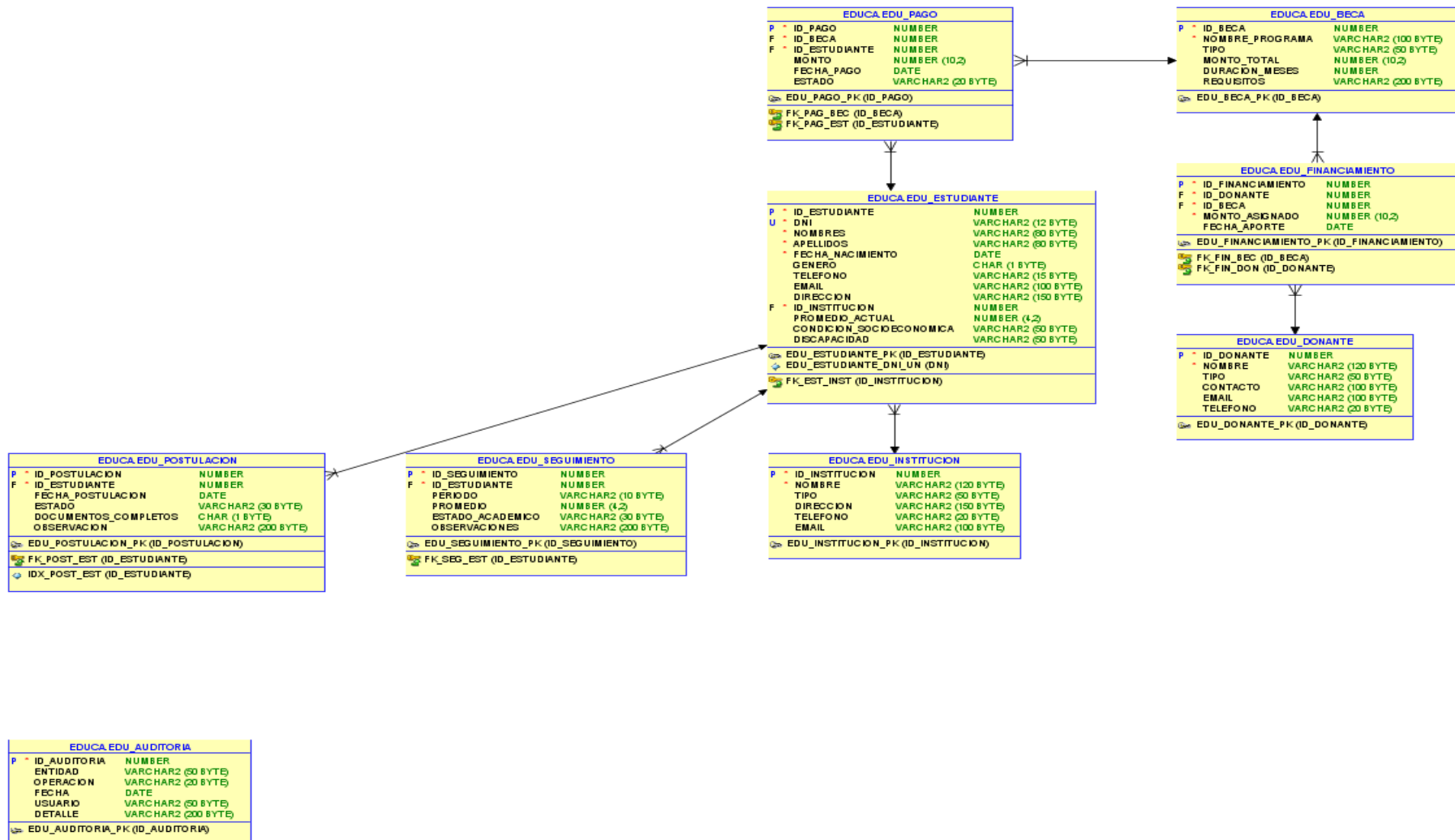
		RN-43).
EDU_BECA	Define el catálogo de becas disponibles (oferta), incluyendo requisitos, montos máximos y alcance.	Sirve como catálogo para la tabla EDU_FINANCIAMIENTO y como base para la asignación individualizada.
EDU_FINANCIAMIENTO	Registra la asignación de fondos del EDU_DONANTE hacia un programa específico de EDU_BECA.	Actúa como tabla transaccional (N:M) que asegura que toda asignación de beca esté condicionada a fondos disponibles, cumpliendo con la Regla RN-23.
EDU_PAGO	Registra la ejecución financiera de la beca, proporcionando trazabilidad de los desembolsos.	Se relaciona directamente con la beca asignada (ID_BECA), lo que permite generar reportes de ejecución (RN-61).
EDU_AUDITORIA	Implementa el requisito de Trazabilidad Obligatoria.	Su diseño registra la entidad_afectada, acción, usuario y el estado antes/después, cumpliendo con los estándares de control

		de calidad.
--	--	-------------

Nota: El Diseño o Modelo Físico de la Base de Datos del Proyecto “Sistema Fundación Educa Perú”, garantiza la integridad referencial y la coherencia de los datos, reflejando las cardinalidades definidas en el modelo conceptual.

- **Relación 1:N:** Evidente en la mayoría de las uniones, por ejemplo: una EDU_INSTITUCION puede tener “N” EDU_ESTUDIANTE.
- **Relación de Proceso (1:N):** La estructura garantiza que un estudiante (EDU_ESTUDIANTE) pueda tener múltiples registros de EDU_POSTULACION, EDU_PAGO y EDU_SEGUIMIENTO a lo largo del tiempo.
- **Restricciones de Integridad:** Se han configurado Foreign Keys (FK) para asegurar que no se pueda registrar una postulación sin un estudiante válido o un pago sin una beca asignada existente.

Diagrama del Modelo de Datos Físico



8. Implementación de la Lógica de Negocio y Transaccionalidad

8.1. Centralización de la Lógica de Negocio (PL/SQL)

Para garantizar la integridad de los datos y centralizar las reglas de negocio del sistema Fundación Educa Perú, se implementó el paquete de base de datos EDU_PKG_GESTION. Este componente encapsula los procedimientos críticos, evitando que las aplicaciones cliente manipulen directamente las tablas y asegurando que siempre se cumplan las validaciones.

A continuación, se detalla la especificación técnica de los procedimientos almacenados desarrollados:

Tabla 04: Especificación Técnica: REGISTRAR_POSTULACION

Característica	Descripción
Objetivo	Automatizar la creación de una solicitud inicial verificando la identidad del alumno en la base de datos maestra.
Parámetros de Entrada	p_dni (VARCHAR2): Documento de identidad del estudiante.
Lógica de Negocio	1. Busca el ID_ESTUDIANTE asociado al DNI ingresado
	2. Si existe, inserta un nuevo registro en EDU_POSTULACION.
	3. Inicializa el estado en 'PENDIENTE' y la bandera de documentos en 'N'.
Excepciones	ORA-20001: "El estudiante con DNI [...] no existe." (Impide postulaciones de personas no registradas).

Tabla 05: Especificación Técnica: ASIGNAR_BECA (Core del Negocio)

Característica	Descripción
Objetivo	Procesar la asignación de una beca asegurando la disponibilidad de fondos (Integridad Financiera) y manejando la concurrencia.
Parámetros de Entrada	• p_id_postulacion (NUMBER) • p_monto_mensual (NUMBER) • p_usuario_auditor (VARCHAR2)

Lógica Transaccional	1. Identificación: Localiza el financiamiento más antiguo que tenga SALDO_DISPONIBLE suficiente.
	2. Bloqueo Pesimista: Aplica FOR UPDATE sobre ese registro financiero específico para evitar conflictos de concurrencia.
	3. Descuento: Resta el p_monto_mensual al saldo del financiamiento.
	4. Actualización: Cambia el estado de la postulación a 'APROBADA'.
	5. Programación: Genera el primer pago en estado 'PENDIENTE' en la tabla EDU_PAGO.
	6. Auditoría: Inserta el registro de la operación en EDU_AUDITORIA.
Excepciones	ORA-20002: "No hay fondos disponibles suficientes." (Se dispara si ningún donante tiene saldo para cubrir el monto).

Evidencia de Ejecución Exitosa: En la siguiente captura se observa la ejecución del flujo completo: registro de postulación y asignación de beca, donde el sistema descontó correctamente los fondos y generó el pago pendiente.

```

SET SERVEROUTPUT ON;
DECLARE
    v_dni VARCHAR2(15) := '12345678'; -- El alumno que creamos
    v_id_postulacion NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- 1. REGISTRANDO POSTULACIÓN ---');
    EDU_PKG_GESTION.REGISTRAR_POSTULACION(v_dni);

    -- Obtenemos el ID generado para usarlo en el siguiente paso
    SELECT MAX(ID_POSTULACION) INTO v_id_postulacion FROM EDU_POSTULACION;
    DBMS_OUTPUT.PUT_LINE('Postulación creada con ID: ' || v_id_postulacion);

    DBMS_OUTPUT.PUT_LINE('--- 2. EJECUTANDO ASIGNACIÓN DE BECA ---');
    -- Intentamos asignar una beca de 1,000 soles mensuales
    EDU_PKG_GESTION.ASIGNAR_BECA(
        p_id_postulacion => v_id_postulacion,
        p_monto_mensual => 1000,
        p_usuario_auditor => 'COORD_BECAS'
    );

    DBMS_OUTPUT.PUT_LINE('¡Éxito! Beca asignada y fondos descontados.');
```

END;

/

Resultado de la Consulta x Salida de Script x

Tarea terminada en 0.548 segundos

Package Body EDU_PKG_GESTION compilado

```

--- 1. REGISTRANDO POSTULACIÓN ---
Postulación creada con ID: 4
--- 2. EJECUTANDO ASIGNACIÓN DE BECA ---
¡Éxito! Beca asignada y fondos descontados.

Procedimiento PL/SQL terminado correctamente.
```

Tabla 06: Especificación Técnica: REGISTRAR_SEGUIMIENTO

Característica	Descripción
Objetivo	Registrar el desempeño académico periódico para evaluar la continuidad del beneficio.
Parámetros de Entrada	• p_dni (VARCHAR2) • p_periodo (VARCHAR2) • p_promedio (NUMBER)
Lógica de Negocio	1. Recupera el ID del estudiante.
	2. Evalúa automáticamente el estado académico: - Si p_promedio >= 13 → Estado 'APROBADO' - Caso contrario → Estado 'OBSERVADO'
	3. Registra la evaluación en la tabla EDU_SEGUIMIENTO.

8.2. Estrategia de Control de Concurrency

La integridad financiera es el requisito no funcional más crítico del sistema (Regla RN-23). En un entorno de producción con múltiples usuarios concurrentes (Coordinadores de Becas), existe el riesgo técnico de "Condición de Carrera" (Race Condition).

8.2.1. Escenario de Riesgo (Doble Gasto)

Se analizó el escenario donde dos transacciones intentan comprometer el mismo fondo simultáneamente. Sin un control de concurrencia, ocurriría la siguiente inconsistencia:

1. **Estado Inicial:** El fondo F01 tiene un Saldo Disponible de S/ 1,000.
2. **Tiempo T1:** El Coordinador A lee el saldo (Ve S/ 1,000).
3. **Tiempo T2:** El Coordinador B lee el mismo saldo (Ve S/ 1,000) antes de que A termine.
4. **Tiempo T3:** Ambos proceden a asignar una beca de S/ 1,000.
5. **Resultado Fatal:** El sistema permite ambas operaciones y el saldo final queda en -S/ 1,000 (Saldo Negativo), violando la integridad del negocio.

8.2.2. Solución Técnica: Bloqueo Pesimista

Para mitigar este riesgo, se implementó una estrategia de Bloqueo Pesimista utilizando la cláusula `SELECT ... FOR UPDATE` de Oracle. Esto obliga a la base de datos a bloquear físicamente la fila del financiamiento seleccionado hasta que la transacción termine (`COMMIT` o `ROLLBACK`).

Implementación en PL/SQL: Debido a la restricción **ORA-02014** (que impide usar `FOR UPDATE` junto con `ORDER BY` y `FETCH FIRST` en la misma sentencia), se diseñó una lógica de dos pasos dentro del paquete `EDU_PKG_GESTION`:

1. **Identificación:** Se selecciona el ID del financiamiento más antiguo con saldo (lectura sin bloqueo).
2. **Bloqueo Efectivo:** Se realiza una segunda consulta directa por ese ID aplicando el bloqueo.

A continuación, se presenta el fragmento del código fuente implementado que garantiza la serialización de estas transacciones:

```
-- Paso 1: Identificar el financiamiento disponible (Lectura)
BEGIN
    SELECT ID_FINANCIAMIENTO
    INTO v_id_fin
    FROM (
        SELECT ID_FINANCIAMIENTO
        FROM EDU_FINANCIAMIENTO
        WHERE ID_BECA = v_id_beca AND SALDO_DISPONIBLE >= p_monto_mensual
        ORDER BY FECHA_APORTE ASC
    )
    WHERE ROWNUM = 1; -- Tomamos solo el primero
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20002, 'No hay fondos disponibles suficientes.');
```

```
-- Paso 2: Aplicar el candado (Concurrencia)
-- Si otra sesión tiene esta fila, el sistema ESPERA aquí.
SELECT SALDO_DISPONIBLE
INTO v_saldo
FROM EDU_FINANCIAMIENTO
WHERE ID_FINANCIAMIENTO = v_id_fin
FOR UPDATE; -- <--- Bloqueo Pesimista para evitar doble gasto
```

Resultado: Si el Coordinador B intenta acceder al mismo fondo mientras el Coordinador A está procesando, la sesión de B entrará en estado de espera (WAIT) automáticamente, garantizando que nunca se gaste dinero que no existe.

8.3. Gestión de Transacciones y Consistencia (ACID)

El sistema garantiza la Atomicidad y Consistencia de las operaciones complejas. Esto significa que un proceso de negocio (como asignar una beca o registrar un pago) se trata como una unidad indivisible: o se ejecutan todos sus pasos correctamente, o no se ejecuta ninguno.

Para lograr esto, se implementó el uso de **SAVEPOINT** y **ROLLBACK** dentro de los bloques PL/SQL. Esta estrategia permite manejar excepciones controladas sin perder el trabajo previo válido (como registros de auditoría o trazas de inicio).

8.3.1. Mecanismo de Recuperación ante Fallos

La lógica implementada sigue el siguiente flujo estándar:

1. **Inicio de Transacción:** Se registra una auditoría de inicio (operación válida).
2. **Punto de Guardado:** Se establece un **SAVEPOINT punto_seguro**.
3. **Operación Crítica:** Se intenta ejecutar la lógica de negocio (ej. insertar un pago).
4. **Validación:** Si se detecta una violación de regla de negocio (ej. monto negativo o restricción de integridad), el sistema captura la excepción.
5. **Recuperación:** Se ejecuta un **ROLLBACK TO punto_seguro**, deshaciendo solo la operación errónea pero manteniendo el registro de auditoría inicial.

8.3.2. Prueba de Transaccionalidad

Se realizó una prueba de estrés simulando la inserción de un pago con un monto inválido (-500). Como se evidencia a continuación, el sistema detectó la violación de la restricción, generó una alerta y revirtió la operación defectuosa automáticamente.

```

SET SERVEROUTPUT ON;
DECLARE
    v_id_beca NUMBER := 1;
    v_id_est NUMBER := 1;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- INICIO DE DEMOSTRACIÓN DE TRANSACCIONES ---');

    -- 1. Operación válida: Registramos una auditoria de inicio
    -- Esto Sí queremos que se guarde.
    INSERT INTO EDU_AUDITORIA (ID_AUDITORIA, ENTIDAD, OPERACION, DETALLE)
    VALUES (EDU_SEQ_AUDITORIA.NEXTVAL, 'DEMO_TRX', 'INICIO', 'Iniciando proceso de pagos');

    -- Creamos un punto de restauración aquí
    SAVEPOINT punto_seguro;
    DBMS_OUTPUT.PUT_LINE('Paso 1 completado. SAVEPOINT creado.');
```

```

    -- 2. Operación Errónea: Intentamos registrar un pago con monto negativo
    DBMS_OUTPUT.PUT_LINE('Intentando registrar pago con monto inválido (-500)...');

    BEGIN
        INSERT INTO EDU_PAGO (ID_PAGO, ID_BECA, ID_ESTUDIANTE, MONTO, ESTADO)
        VALUES (EDU_SEQ_PAGO.NEXTVAL, v_id_beca, v_id_est, -500, 'ERROR');
```

Salida de Script x Resultado de la Consulta x

Tarea terminada en 0.373 segundos

```

--- INICIO DE DEMOSTRACIÓN DE TRANSACCIONES ---
Paso 1 completado. SAVEPOINT creado.
Intentando registrar pago con monto inválido (-500)...
¡ALERTA DETECTADA!: ORA-02290: restricción de control (EDUCA.CHK_PAG_ESTADO) violada
>>> Ejecutando ROLLBACK TO punto_seguro...
--- FIN DE TRANSACCIÓN: Se guardó la auditoría, se descartó el error. ---
Procedimiento PL/SQL terminado correctamente.
```

8.4. Seguridad: Prevención de Inyección SQL

Dado que el sistema gestiona información sensible (datos personales de estudiantes y montos financieros), se prioriza la seguridad en el desarrollo de la capa de datos. Se implementaron prácticas de codificación segura para mitigar vulnerabilidades críticas, específicamente la **Inyección SQL**.

8.4.1. Análisis de Vulnerabilidad

Para validar la seguridad del sistema, se desarrollaron dos versiones de un procedimiento de búsqueda de estudiantes y se sometieron a pruebas de penetración (Ethical Hacking básico):

1. **Versión Vulnerable (Mala Práctica):** Utiliza concatenación de cadenas (||) para construir la consulta SQL dinámicamente.
 - **Riesgo:** Un atacante puede inyectar código malicioso (ej. ' OR '1'='1) para manipular la lógica de la consulta y acceder a todos los registros de la base de datos sin autorización.
2. **Versión Segura (Implementada):** Utiliza Bind Variables (parámetros vinculados).
 - **Solución:** Oracle separa estrictamente la sentencia SQL de los datos. El input del usuario se trata como un valor literal y nunca como código ejecutable.

8.4.2. Evidencia de Pruebas de Seguridad

A continuación, se presenta la evidencia de la prueba realizada. Se intentó vulnerar ambos procedimientos inyectando el patrón ' OR '1'='1.

- **Resultado Vulnerable:** El sistema expuso indebidamente la lista completa de estudiantes.
- **Resultado Seguro:** El sistema neutralizó el ataque, buscando literalmente el texto ingresado y no devolviendo ningún resultado, garantizando la confidencialidad de la información.

```
SET SERVEROUTPUT ON;
BEGIN
  DBMS_OUTPUT.PUT_LINE('--- PRUEBA DE SEGURIDAD (INYECCIÓN SQL) ---');

  DBMS_OUTPUT.PUT_LINE('1. Intento de hackeo en procedimiento INSEGURO:');
  -- El truco ' OR '1'='1' engaña al código para mostrar TODOS los alumnos
  BUSCAR_ALUMNO_INSEGURO('' OR '1'='1');

  DBMS_OUTPUT.PUT_LINE('-----');

  DBMS_OUTPUT.PUT_LINE('2. Intento de hackeo en procedimiento SEGURO:');
  -- Aquí Oracle busca literalmente un DNI llamado " OR '1'='1", que no existe.
  BUSCAR_ALUMNO_SEGURO('' OR '1'='1');

  DBMS_OUTPUT.PUT_LINE('--- FIN DE PRUEBA ---');
END;
/
```

Salida de Script x Resultado de la Consulta x

Tarea terminada en 0.608 segundos

```
--- PRUEBA DE SEGURIDAD (INYECCIÓN SQL) ---
1. Intento de hackeo en procedimiento INSEGURO:
[VULNERABLE] Alumno encontrado: Juan
-----
2. Intento de hackeo en procedimiento SEGURO:
--- FIN DE PRUEBA ---

Procedimiento PL/SQL terminado correctamente.
```

9. Manejo de Seguridad, auditoría, backup, restore y gestión de usuarios

La seguridad de la información es crítica para un sistema que maneja datos personales, financieros y académicos. Aquí se definen políticas técnicas y operativas para proteger confidencialidad, integridad y disponibilidad.

Gestión de contraseñas

Profile de contraseñas (ejemplo EDU_PROFILE) para hacer cumplir caducidad y bloqueos:

- **Profile de contraseñas**
- **Asignar profile a cuentas**
- **Políticas:**

Principio de mínimo privilegio

- Asignar solo los privilegios necesarios por rol.
- Revisar concesiones periódicamente (cada 3 meses).
- Uso de vistas o procedimientos en lugar de conceder SELECT/UPDATE directo sobre tablas críticas.

Protección de información sensible

Datos sensibles deben estar:

- Cifrados por columna o tablespace encryption (TDE) de Oracle.
- **En tránsito:** todas las conexiones a la BD (clientes, aplicaciones) deben usar TLS (SSL).
- **Enmascaramiento / Reducción:**
 - En ambientes de desarrollo/test usar datos enmascarados o sintéticos.
- **Control de acceso:**
 - Roles definidos y controlados.
 - Acceso externo solo mediante API/Bastion host con autenticación y registro.
- **Registro y monitoreo:**
 - Uso de EDU_AUDITORIA + logs del motor para detectar accesos no autorizados.

De esta manera definimos nuestro plan a seguir:

Roles y usuarios

9.1 Roles previstos

- **ROLE_ADMIN_BD** : administración completa del esquema/BD (solo para DBAs).
- **ROLE_COORD_BECAS** : funciones de coordinación de becas (asignación, modificación, visualización y generación de pagos).
- **ROLE_ANALISTA** : acceso para consultas y reportes (solo SELECT; acceso a la tabla de auditoría en lectura).

Script: creación de roles y privilegios en Oracle

```
-- 1) Crear roles

CREATE ROLE ROLE_ADMIN_BD;

CREATE ROLE ROLE_COORD_BECAS;

CREATE ROLE ROLE_ANALISTA;


-- 2) Privilegios de sistema

-- ROLE_ADMIN_BD: privilegios elevados (ejecutar por DBA)
```



```
GRANT CREATE SESSION TO ROLE_ADMIN_BD;

GRANT CREATE TABLE, CREATE VIEW, CREATE PROCEDURE, CREATE
SEQUENCE, CREATE TRIGGER TO ROLE_ADMIN_BD;

GRANT DROP ANY TABLE, ALTER ANY TABLE, DROP ANY PROCEDURE, SELECT
ANY TABLE, INSERT ANY TABLE, UPDATE ANY TABLE, DELETE ANY TABLE
TO ROLE_ADMIN_BD;

-- ROLE_COORD_BECAS

GRANT CREATE SESSION TO ROLE_COORD_BECAS;

GRANT SELECT, INSERT, UPDATE ON EDU_ESTUDIANTE TO
ROLE_COORD_BECAS;

GRANT SELECT, INSERT, UPDATE ON EDU_POSTULACION TO
ROLE_COORD_BECAS;

GRANT SELECT, INSERT, UPDATE ON EDU_BECA_ASIGNADA TO
ROLE_COORD_BECAS;

GRANT SELECT, INSERT, UPDATE ON EDU_PAGO_DESembolso TO
ROLE_COORD_BECAS;

GRANT EXECUTE ON EDU_PKG_BECA TO ROLE_COORD_BECAS;

-- paquete de asignación de becas

-- ROLE_ANALISTA: solo lectura sobre datos y auditoría

GRANT CREATE SESSION TO ROLE_ANALISTA;

GRANT SELECT ON EDU_ESTUDIANTE TO ROLE_ANALISTA;

GRANT SELECT ON EDU_PROGRAMA_BECA TO ROLE_ANALISTA;

GRANT SELECT ON EDU_FINANCIAMIENTO TO ROLE_ANALISTA;

GRANT SELECT ON EDU_BECA_ASIGNADA TO ROLE_ANALISTA;

GRANT SELECT ON EDU_PAGO_DESembolso TO ROLE_ANALISTA;

GRANT SELECT ON EDU_AUDITORIA TO ROLE_ANALISTA;
```

Usuarios de ejemplo y asignación de roles

```
-- Crear usuarios

CREATE USER admin_beca IDENTIFIED BY "Adm!n2025" DEFAULT
TABLESPACE USERS QUOTA UNLIMITED ON USERS;

CREATE USER coord_becas IDENTIFIED BY "Coord2025!" DEFAULT
TABLESPACE USERS QUOTA 50M ON USERS;

CREATE USER analista_report IDENTIFIED BY "Anal!sta2025" DEFAULT
TABLESPACE USERS QUOTA 20M ON USERS;

-- Asignar roles

GRANT ROLE_ADMIN_BD TO admin_beca;

GRANT ROLE_COORD_BECAS TO coord_becas;

GRANT ROLE_ANALISTA TO analista_report;

-- Permisos adicionales para administración de usuarios

GRANT CREATE SESSION, CREATE VIEW TO admin_beca;
```

9.2 Auditoría

La tabla **EDU_AUDITORIA** se diseñó para registrar: entidad afectada, id de la entidad, acciones, usuario, fecha/hora, estado antes/después y motivo.

```
CREATE TABLE EDU_AUDITORIA (

  ID_AUD          NUMBER(10) PRIMARY KEY,

  ENTIDAD         VARCHAR2(100),

  ENTIDAD_ID      NUMBER(10),

  ACCION          VARCHAR2(30),

  USUARIO         VARCHAR2(200),

  FECHA_HORA      TIMESTAMP DEFAULT SYSTIMESTAMP,
```

```

    ANTES          CLOB,

    DESPUES        CLOB,

    MOTIVO          VARCHAR2(400)

);

```

9.3 Triggers Auditoría

EDU_BECA_ASIGNADA y EDU_PAGO_DESembolso

```

CREATE OR REPLACE TRIGGER trg_audit_beca_asignada

AFTER INSERT OR UPDATE OR DELETE ON EDU_BECA_ASIGNADA

FOR EACH ROW

DECLARE

    v_before CLOB;

    v_after  CLOB;

    v_user   VARCHAR2(200) :=
SYS_CONTEXT('USERENV','SESSION_USER');

BEGIN

    IF INSERTING THEN

        v_after := row_to_clob('EDUCA','EDU_BECA_ASIGNADA','ID_BECA',
:NEW.ID_BECA);

        INSERT INTO EDU_AUDITORIA (ID_AUD, ENTIDAD, ENTIDAD_ID,
ACCION, USUARIO, ANTES, DESPUES, MOTIVO)

        VALUES (EDU_SEQ_AUD.NEXTVAL, 'EDU_BECA_ASIGNADA',
:NEW.ID_BECA, 'INSERT', v_user, NULL, v_after, NULL);

    ELSIF UPDATING THEN

        v_before :=
row_to_clob('EDUCA','EDU_BECA_ASIGNADA','ID_BECA', :OLD.ID_BECA);

        v_after  :=
row_to_clob('EDUCA','EDU_BECA_ASIGNADA','ID_BECA', :NEW.ID_BECA);

```

```

        INSERT INTO EDU_AUDITORIA (ID_AUD, ENTIDAD, ENTIDAD_ID,
ACCION, USUARIO, ANTES, DESPUES, MOTIVO)

        VALUES (EDU_SEQ_AUD.NEXTVAL, 'EDU_BECA_ASIGNADA',
:NEW.ID_BECA, 'UPDATE', v_user, v_before, v_after, NULL);

    ELSIF DELETING THEN

        v_before :=
row_to_clob('EDUCA','EDU_BECA_ASIGNADA','ID_BECA', :OLD.ID_BECA);

        INSERT INTO EDU_AUDITORIA (ID_AUD, ENTIDAD, ENTIDAD_ID,
ACCION, USUARIO, ANTES, DESPUES, MOTIVO)

        VALUES (EDU_SEQ_AUD.NEXTVAL, 'EDU_BECA_ASIGNADA',
:OLD.ID_BECA, 'DELETE', v_user, v_before, NULL, NULL);

    END IF;

EXCEPTION

    WHEN OTHERS THEN

        -- No bloquear la transacción del negocio por errores en
auditoría: registrar en log alternativo si se desea.

        NULL;

END;

/

```

```

CREATE OR REPLACE TRIGGER trg_audit_pago_desembolso

AFTER INSERT OR UPDATE OR DELETE ON EDU_PAGO_DESembolso

FOR EACH ROW

DECLARE

    v_before CLOB;

    v_after  CLOB;

    v_user   VARCHAR2(200) :=
SYS_CONTEXT('USERENV','SESSION_USER');

```

```

BEGIN

    IF INSERTING THEN

        v_after :=
row_to_clob('EDUCA','EDU_PAGO_DESembolso','ID_PAGO',
:NEW.ID_PAGO);

        INSERT INTO EDU_AUDITORIA (ID_AUD, ENTIDAD, ENTIDAD_ID,
ACCION, USUARIO, ANTES, DESPUES, MOTIVO)

        VALUES (EDU_SEQ_AUD.NEXTVAL, 'EDU_PAGO_DESembolso',
:NEW.ID_PAGO, 'INSERT', v_user, NULL, v_after, NULL);

    ELSIF UPDATING THEN

        v_before :=
row_to_clob('EDUCA','EDU_PAGO_DESembolso','ID_PAGO',
:OLD.ID_PAGO);

        v_after :=
row_to_clob('EDUCA','EDU_PAGO_DESembolso','ID_PAGO',
:NEW.ID_PAGO);

        INSERT INTO EDU_AUDITORIA (ID_AUD, ENTIDAD, ENTIDAD_ID,
ACCION, USUARIO, ANTES, DESPUES, MOTIVO)

        VALUES (EDU_SEQ_AUD.NEXTVAL, 'EDU_PAGO_DESembolso',
:NEW.ID_PAGO, 'UPDATE', v_user, v_before, v_after, NULL);

    ELSIF DELETING THEN

        v_before :=
row_to_clob('EDUCA','EDU_PAGO_DESembolso','ID_PAGO',
:OLD.ID_PAGO);

        INSERT INTO EDU_AUDITORIA (ID_AUD, ENTIDAD, ENTIDAD_ID,
ACCION, USUARIO, ANTES, DESPUES, MOTIVO)

        VALUES (EDU_SEQ_AUD.NEXTVAL, 'EDU_PAGO_DESembolso',
:OLD.ID_PAGO, 'DELETE', v_user, v_before, NULL, NULL);

    END IF;

EXCEPTION

    WHEN OTHERS THEN

```

```
NULL;  
  
END;  
  
/
```

9.4 Consultas de Auditoría:

Consultas que permiten revisar la auditoría por entidad, usuario, fechas y acción.

- **Auditoría reciente (últimos 7 días):**

```
SELECT ID_AUD, ENTIDAD, ENTIDAD_ID, ACCION, USUARIO,  
FECHA_HORA  
  
FROM EDU_AUDITORIA  
  
WHERE FECHA_HORA >= SYSTIMESTAMP - INTERVAL '7' DAY  
  
ORDER BY FECHA_HORA DESC;
```

- **Historial de cambios para una beca (id = 123):**

```
SELECT ID_AUD, ACCION, USUARIO, FECHA_HORA, ANTES, DESPUES,  
MOTIVO  
  
FROM EDU_AUDITORIA  
  
WHERE ENTIDAD = 'EDU_BECA_ASIGNADA' AND ENTIDAD_ID = 123  
  
ORDER BY FECHA_HORA DESC;
```

9.5 Estrategia de backup & restore

Política propuesta

Tipo/ frecuencia:

- Backup full completo (imagen): semanal (domingo noche).
- Backups incrementales: diarios (nivel 1) cada noche.

- Archivelogs: activados y archivados continuamente (si BD en modo ARCHIVELOG).
- Backup offsite/replicación: copia mensual cifrada fuera de sitio.

Cifrado: dumps y backups cifrados (ENCRYPTION) y acceso restringido.

Pruebas: restauración probada cada mes en entorno de DR o pre-producción.

Responsables: DBA

Export schema completo (EDUCA):

```
# Export schema EDUCA (full schema dump)

expdp system/oracle PASSWORD=tu_pass DIRECTORY=DATA_PUMP_DIR
DUMPFILE=educa_schema_full_%U.dmp LOGFILE=educa_exp_full.log
SCHEMAS=EDUCA PARALLEL=4 ENCRYPTION=ALL
ENCRYPTION_PASSWORD=EncPass2025
```

Procedimiento mínimo de restore con expdp

```
impdp system/tu_pass DIRECTORY=DATA_PUMP_DIR
DUMPFILE=educa_schema_full_01.dmp LOGFILE=restore_imp.log FULL=Y
```